

Name: Jiecheng Zhu

Educational Background

Bachelor's degree: University of California, Santa Cruz

Bachelor's degree in Computer Science (Graduated in June 2025).

Master's degree: Boston University

Master's degree in Applied Business Analytics. Currently studying Machine Learning and Deep Learning (AI), expected to graduate in Q2 2027.

Computer Software Professional Skills

- Data Structures and Algorithms: Linked Lists, Stacks, Queue/Priority Queue, Hash Tables, Tree (Pre/In/Post/Level-Order Traversal Algorithms, BSTree, AVL, RBTre), Graph (DFS, BFS, Shortest Path, A*), Sort (Quick Sort, Merge Sort, Heap Sort), Binary Search, Two Pointers, Prefix Sum, Sliding Window, Divide and Conquer, Dynamic Programming/DP, Backtrack
- Programming languages: C/C++, Python, SQL, JavaScript, Java, HTML, R, Haskell. Over 10,000 lines of C/C++ development experience.

Software Development Project Experience

Project 1: HTTP Server

01/2024 – 03/2024

- I developed a simplified multithreaded HTTP server using ANSI C sockets, which can respond to HTTP GET/PUT requests from multiple clients simultaneously.
- For each new client, the HTTP server creates a thread to communicate with it.
- Use semaphores and read-write locks to protect HTTP server-side file resources from concurrent PUT write access by multiple clients.
- It enables file transfer between the client and server, and the client can also send real-time communication messages to the server.

Project 2: Distributed OS Multi-Server Backup System

04/2024 – 06/2024

- Based on vector-clock, KVS data synchronization between the three servers is achieved, making them appear as a single server to the outside world. Vector-clock is formed by merging the logical clocks of the three servers. Each server has its own logical clock, with the count starting from 0.
- Each time the server receives a SET key-value message from the client, it increments its logical clock counter by 1 and updates its local vector clock. The server then synchronizes the updated key-value and the latest local vector clock with other servers and also returns relevant information to the client.
- When a client retrieves a new value for a key from a server, it includes the vector clock associated with that key, stored locally on the client's end. If the server's vector clock for the corresponding key-value pair is older than the client's vector clock, the server will not respond to the client's request until it synchronizes a new value from other servers that meets the requirements. This ensures that the causal timing of events is maintained, which is the requirement of a distributed operating system for maintaining the causal relationship of logical clocks.
- The server is implemented based on FastAPI, and uses httpx to handle client/server communication and communication between servers. Coroutines are used to improve the server's asynchronous response performance.

Project 3: Full-Stack Storage System (React + Express + MongoDB)

07/2024 – 09/2024

- A key-value storage system based on a React frontend and an Express + MongoDB backend has been implemented.
- It supports adding, querying, and deleting key-value pairs.
- Swagger provides RESTful API documentation, offering a good user experience and user-friendly development interface.
 - Frontend: React component-based design
 - Backend: Node.js + Express + MongoDB
 - Database: MongoDB, local or remote connection
 - API Documentation: Provides a visual, interactive interface using swagger-jsdoc and swagger-ui.

Project 4: Path Planning and Navigation System Based on A* Algorithm

01/2025 – 03/2025

- In the Unity environment, a smart car game system with path planning and navigation was implemented based on the A* algorithm.
- The system covers the entire process from navigation map grid generation and A* path planning to autonomous motion control of the game car, and is suitable for scenarios such as game AI and robot simulation.
 - Use a bisection method to divide the polygon map into multiple convex polygon grids.
 - This system uses the A* algorithm to plan the path for an intelligent vehicle from its starting point to its target location. First, the vehicle moves to the center of its current grid cell, then to the midpoint of the common edge between its current and the next grid cell, and then to the center of the next grid cell. Once it reaches the center of the target grid cell, it moves to the target location. Since all grid cells are convex polygons, this movement method ensures that the intelligent vehicle does not move outside the boundaries of the original map.